# When Does Streaming Tool Use Help?

Characterizing Tool-Intent Stabilization in Streaming Retrieval-Augmented Generation

Elroy Galbraith, PhD
SMG Labs

June 17, 2026

**Abstract**

Streaming Retrieval-Augmented Generation (Streaming RAG) reduces user-perceived latency by issuing tool queries in parallel with ongoing user input, before the utterance is complete. Reported gains are aggregate, yet the mechanism's benefit is fundamentally *query-intrinsic*: speculation can only help when the correct tool query becomes determinable before the user stops speaking or typing. We isolate and measure this property—**tool-intent stabilization**, the point in the input stream at which a speculative query's retrieval converges to the answer-bearing result. On the CRAG benchmark (1371 validation questions) we (i) measure the distribution of stabilization, (ii) derive a model-agnostic upper bound on achievable latency savings as a function of tool latency $L$ and input cadence $\delta$, (iii) validate that bound against a working streaming pipeline, and (iv) identify which query properties predict early versus late stabilization. The study requires no model training and runs on commodity CPU hardware. We find that sufficiency stabilizes early for most answerable questions (mean $\phi_{\text{suf}} = 0.26$), that a large majority of queries (73.9%) admit substantial latency hiding at realistic operating points, and that question type is a strong, ordered predictor of stabilization—directly informing when a learned speculative trigger is worth its cost.

## 1 Introduction

Tool-augmented dialogue systems increasingly rely on external retrieval to remain factual, but tool calls add latency that disrupts conversational flow. Streaming RAG [1] addresses this by predicting and issuing tool queries *in parallel with* the user's still-arriving input, then reflecting on whether retrieved results suffice. The approach reports sizable accuracy and latency improvements and is modality-agnostic, applying equally to typed input.

The reported latency benefit, however, is an aggregate over a benchmark. It leaves a basic question unanswered: **for which queries can speculation help at all, and by how much?** The answer is not a property of the model, the retriever, or the speech stack—it is a property of *where, within an utterance, the information that determines the tool query first appears.* If the decisive term arrives last ("who makes the console called the *PlayStation*"), no early speculative query can retrieve the right evidence and the latency win collapses to zero regardless of system quality. If the decisive term arrives early, almost the entire tool latency can be hidden behind the remaining input.

We study this directly. By characterizing tool-intent stabilization as a measurable, query-intrinsic quantity, we obtain (i) a predictive upper bound on achievable latency savings *before* any system is built; (ii) a principled account of the failure mode; (iii) actionable guidance on whether a learned trigger is worth its cost; and (iv) results that transfer across text and speech because the quantity is modality-agnostic. Our contribution is measurement and analysis rather than a new system.

## 2 Background and Related Work

Retrieval-Augmented Generation [4] grounds generation in retrieved evidence; dense passage retrieval [2] and the sparse BM25 model [5] are the standard retrieval backbones. Streaming RAG [1] adds a speculative dimension: a *Trigger* decides when to fire a tool query from a partial utterance, multiple speculative *Threads* run in parallel, and a *Reflector* judges whether results suffice. The idea is closely related in spirit to speculative decoding [3], which hides latency by doing work that may be discarded; here the discarded work is a premature tool call. Our analysis is orthogonal to the system: we quantify the headroom that any such system can exploit, set by the query alone.

## 3 Problem Formalization

Let a query $Q$ be a token sequence $q_{1:n}$ of length $n$, and let $q_{1:t}$ denote the prefix of the first $t$ tokens. Let $R$ be a retriever returning a ranked document list; write $d_t = \text{top-1}(R(q_{1:t}))$ for the top document retrieved from the prefix, $d_n$ for the full-query top document, and $d^\star$ for the gold answer-bearing document.

**Self-consistency stabilization $t_{\text{sc}}$.** the smallest $t$ such that for all $s \geq t$, $d_s = d_n$: the prefix already retrieves what the full query will.

**Sufficiency stabilization $t_{\text{suf}}$.** the smallest $t$ such that $d^\star \in \text{top-}k(R(q_{1:t}))$: the prefix already surfaces the gold evidence. This is the operationally meaningful notion, since the full query itself may retrieve imperfectly.

**Stabilization fraction $\phi = t^\star/n \in (0, 1]$,** reported for both $t_{\text{sc}}$ and $t_{\text{suf}}$. Lower $\phi$ means earlier stabilization and more headroom to hide latency.

**Stabilization volatility $V$,** the number of top-1 changes occurring *after* $d_n$ is first reached. High $V$ signals early-commit risk: a confident-looking early result that a later token would overturn.

**Hidden latency.**

$$H(Q; L, \delta) = \min\big(L,\ \max(0, (n - t^\star) \cdot \delta)\big), \tag{1}$$

where $L$ is tool latency and $\delta$ is the per-token arrival interval. $H$ is the portion of tool latency that completes behind the user's remaining input—the achievable per-query perceived-latency saving.

**Streamable.** Streamable$(Q; L, \delta, \theta)$ holds iff $H \geq \theta L$ for a coverage threshold $\theta$ (e.g. $\theta = 0.8$ hides at least 80% of tool latency). The streamable fraction of a workload is the central deployment-relevant statistic.

## 4 Method

**Data.** We use CRAG, the Comprehensive RAG Benchmark [6], Task 1&2 dev set: 2706 questions, of which 1371 are the validation split we analyze. Each question carries up to five retrieved web pages, a gold answer (plus alternates), and a question-type label.

**Grounding $d^\star$.** CRAG provides no gold-*passage* label, only gold answer strings. We derive $d^\star$ by grounding the normalized gold answer(s) into passage text: substring match for multi-token answers, and word-boundary match for single-token answers, over the answer and its alternates. Questions whose answer is never a literal span (false-premise, "I don't know", and many aggregation/dynamic answers) are *ungroundable* and excluded from $t_{\text{suf}}$; we report the groundable rate explicitly. Because $t_{\text{sc}}$ requires no grounding, we report it as a grounding-free robustness check alongside $t_{\text{suf}}$.

**Passages and retrieval.** Page HTML is cleaned to text and chunked into overlapping 120-word windows. We index each question's passages with BM25 [5] and retrieve over every prefix $q_{1:t}$, $t = 1 \dots n$, recording $d_t$ and the top-$k$ set at each prefix. We sweep $k \in \{1, 3, 5\}$ for the sufficiency definition.

**Latency model and harness (RQ3).** We stream each query as a uniform word sequence—a controlled proxy for input cadence $\delta$—and sweep $(L, \delta, \theta)$ analytically over the grid $L \in \{100, 300, 600, 1000\}$ ms, $\delta \in \{2, 3, 4\}$ w/s, $\theta \in \{0.5, 0.8, 1.0\}$ to obtain the streamable surface (RQ2). For RQ3 we replay a subset of questions through a faithful asynchronous Streaming RAG harness (fixed-interval Trigger, parallel Threads, Reflector) calibrated with the per-stage latencies reported by Arora et al. [1], and compare the *measured* perceived-latency saving (baseline minus streaming) against the $H$ bound.

# 5 Results

We analyze the 1371 validation questions of CRAG Task 1&2. Under our string-grounding procedure, 35.4% of questions are *groundable* (the gold answer appears verbatim in the retrieved pages); the remainder—dominated by false-premise items and answers that never occur as a literal span (many aggregation and dynamic questions)—are excluded from sufficiency statistics. At $k$=3, BM25 surfaces the gold passage in the top-$k$ at some prefix for 21.3% of all questions. All reported $\phi_{\text{suf}}$ statistics are conditional on this retrieved-gold population; $\phi_{\text{sc}}$ is defined for every question and serves as a grounding-free check.

## 5.1 RQ1: How is stabilization distributed?

The two stabilization notions behave very differently (Fig. 1). Self-consistency stabilizes *late*: the lexical top-1 keeps changing until, on average, $\phi_{\text{sc}}$ reaches mean 0.67 (median 0.73)— roughly three-quarters of the way through the query. Sufficiency, in contrast, stabilizes *early*: the gold passage enters the top-$k$ at mean $\phi_{\text{suf}} = 0.26$ and median 0.14. The gap is the central empirical observation of this study: **the answer-bearing document is retrievable from a short prefix long before the query's lexical top-1 settles**. For streaming tool use this is the favorable regime—one need not wait for the utterance to "finish" for the right evidence to be in hand.

The $\phi_{\text{suf}}$ distribution (Fig. 1) is right-skewed with a large mass near zero and a thinner late tail, rather than cleanly bimodal as H1 anticipated. Volatility is modest: the post-stabilization top-1 changes at all for only 20.9% of questions (mean $V = 0.52$), so early-commit risk is concentrated in a minority of queries rather than pervasive.

## 5.2 RQ4: What predicts early vs. late stabilization?

Question type is a strong predictor of $\phi_{\text{suf}}$, spanning a wide range (Table 1, Fig. 2). However, the ordering refines hypothesis H3 rather than confirming it. H3 expected simple lookups to stabilize early and aggregation/comparison/multi-hop to stabilize late. We instead find *aggregation*

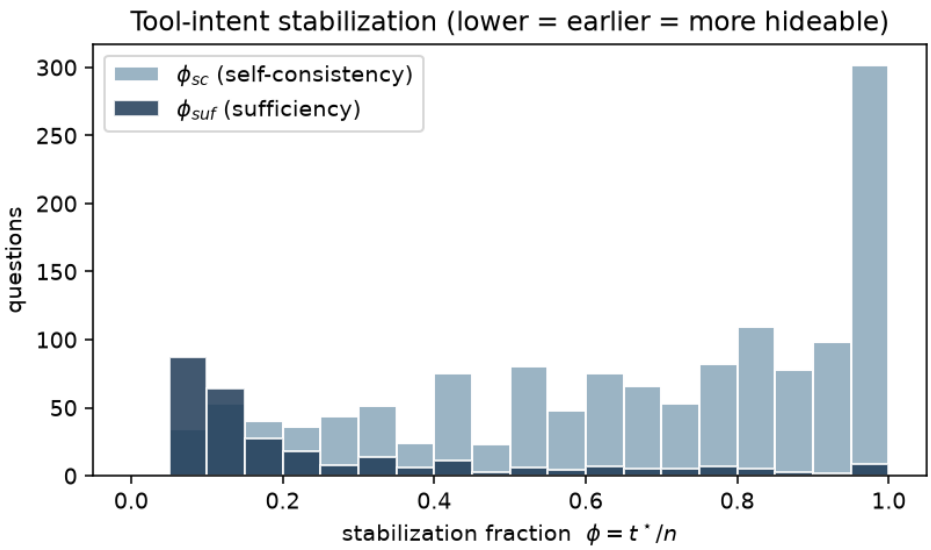


Figure 1: Distribution of stabilization fraction $\phi$ at $k$=3. Sufficiency ($\phi_{\text{suf}}$, dark) concentrates near zero—gold evidence is retrievable early—while self-consistency ($\phi_{\text{sc}}$, light) is late and broad.

Table 1: Sufficiency stabilization by CRAG question type ($k$=3, retrieved-gold questions), sorted earliest to latest.

| question type | $n$ | mean $\phi_{\text{suf}}$ |
|---|---|---|
| aggregation | 32 | 0.198 |
| comparison | 59 | 0.210 |
| multi-hop | 15 | 0.267 |
| simple | 118 | 0.280 |
| false_premise | 9 | 0.289 |
| simple_w_condition | 52 | 0.307 |
| post-processing | 5 | 0.310 |
| set | 2 | 0.658 |

and *comparison* among the *earliest*-stabilizing types, with *set* questions latest. The explanation is that sufficiency measures when the gold *document* becomes retrievable, not when the answer can be *computed*: a comparison or aggregation question often names its key entities up front, so the answer-bearing page is retrievable early even though producing the final answer requires post-retrieval reasoning. In other words, retrieval-sufficiency stabilization is governed by entity position in the utterance, which is not aligned with reasoning complexity. (Several types have small support—`set` $n$=2, `post-processing` $n$=5—so their point estimates are noisy; the larger classes `simple`, `comparison`, and `simple_w_condition` are well populated.)

### 5.3 RQ2: What fraction of queries admit latency hiding?

Using $t^\star = t_{\text{suf}}$ where available and falling back to $t_{\text{sc}}$ otherwise, we compute the streamable fraction over the full $(L, \delta, \theta)$ grid (Table 2, Fig. 3). At the central operating point ($L$=600 ms, $\delta$=3 w/s, $\theta$=0.8), 73.9% of queries admit hiding at least 80% of tool latency. Two patterns stand out. First,

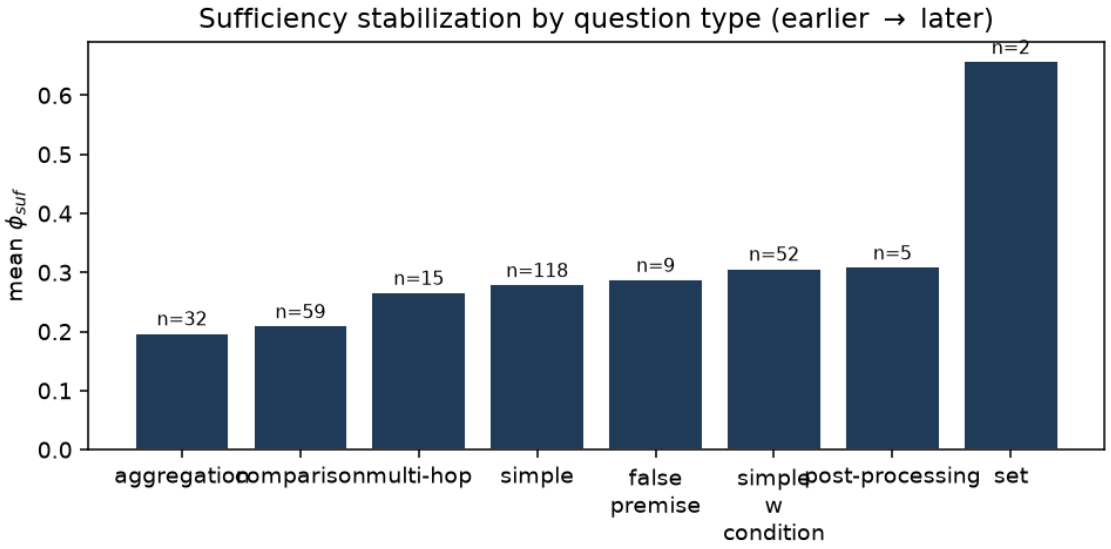


Figure 2: Mean $\phi_{\text{suf}}$ by question type ($k$=3). Type spans a 3× range, but the ordering tracks entity position, not reasoning complexity.

Table 2: RQ2: streamable fraction (%) at $\theta$=0.8 over the $(L, \delta)$ grid. Rows are tool latency $L$ (ms); columns are input cadence $\delta$ (words/sec). For large $L$, slower cadence (lower $\delta$) hides more (H2).

| | $\delta$=2 | $\delta$=3 | $\delta$=4 |
|---|---|---|---|
| $L$=100 | 82.6 | 82.6 | 82.6 |
| $L$=300 | 82.6 | 82.6 | 82.6 |
| $L$=600 | 82.6 | 73.9 | 73.9 |
| $L$=1000 | 73.9 | 63.3 | 54.5 |

for small tool latency ($L \leq 300$ ms) the constraint is essentially never binding: 82.6% of queries are streamable regardless of cadence—the residual input time trivially covers a short tool call. Second, and confirming **H2**, when tool latency is large ($L$=1000 ms) the binding constraint becomes residual input time, so a *slower* cadence *raises* the streamable fraction: from 54.5% at $\delta$=4 w/s up to 73.9% at $\delta$=2 w/s. Faster typists and faster talkers leave less room to hide a slow tool.

### 5.4 RQ3: Does the bound match a working pipeline?

We replayed 60 retrieved-gold questions through the asynchronous Streaming RAG harness at $L$=600 ms, $\delta$=3 w/s. The measured perceived-latency saving (baseline minus streaming) averages 1122 ms, while the $H$ bound predicts 590 ms (Fig. 4). On average, measured savings *exceed* the bound by roughly 1.9× (a small number of queries where the speculative trigger mis-fires save nothing or slightly less than baseline). This is expected and informative: $H$ models only the tool latency hidden behind input, whereas the calibrated pipeline additionally overlaps the LLM's query-generation step ($\approx$ 590 ms [1]) with the user's input. $H$ is therefore a *conservative* predictor of realized savings—it captures the input-hiding component and omits other overlappable costs—so a query that $H$ deems streamable will, in practice, save at least as much as predicted.

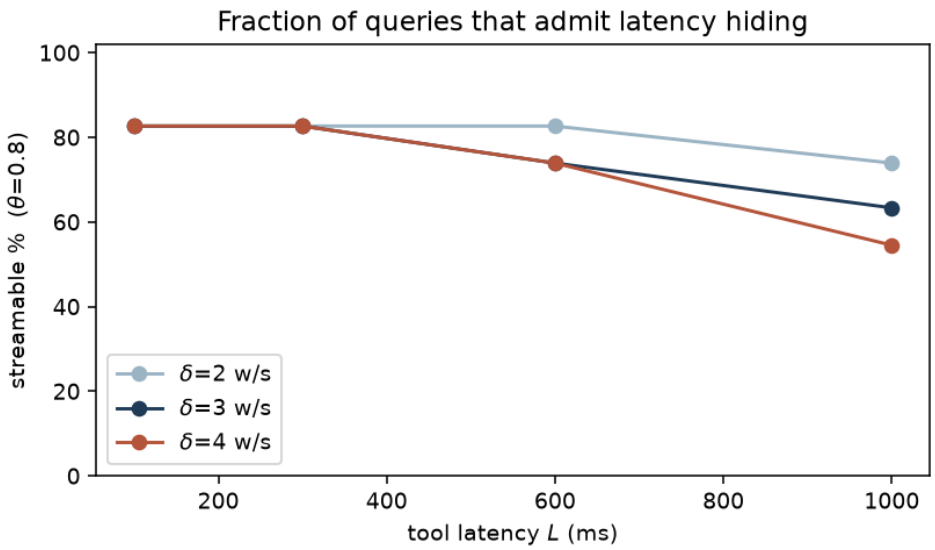


Figure 3: Streamable fraction vs. tool latency $L$ at $\theta$=0.8, one line per cadence $\delta$. The curves separate only once $L$ is large enough for residual input time to bind; there, slower cadence dominates (H2).

Table 3: Top-$k$ robustness. $\phi_{\text{sc}}$ (mean 0.67 / median 0.73) is independent of $k$ and omitted.

| $k$ | retrieved-gold | mean $\phi_{\text{suf}}$ | median $\phi_{\text{suf}}$ |
|---|---|---|---|
| 1 | 15.6% | 0.327 | 0.200 |
| 3 | 21.3% | 0.264 | 0.143 |
| 5 | 25.0% | 0.248 | 0.125 |

### 5.5 Retriever robustness (top-$k$)

Varying the sufficiency top-$k$ shifts absolute values but preserves the qualitative picture (Table 3). Larger $k$ surfaces the gold passage for more questions (retrieved-gold rate 15.6% $\rightarrow$ 25.0%) and stabilizes slightly earlier (mean $\phi_{\text{suf}}$ 0.327 $\rightarrow$ 0.248), as expected since a more permissive top-$k$ is easier to satisfy. The self-consistency curve is unchanged by $k$ (it depends only on the top-1), and the central streamable fraction is stable near 73–74%. The early-sufficiency / late-self-consistency gap holds at every $k$.

## 6 Threats to Validity

**Input timing.** A uniform word stream is a proxy for speech; real ASR pacing is variable and partial hypotheses get revised. We bound this here by using clean text and treat speech timing as planned follow-up.

**Stabilization measure.** Top-1 equality is a coarse proxy for "intent settled." We mitigate by also reporting sufficiency against the gold document and by varying top-$k$.

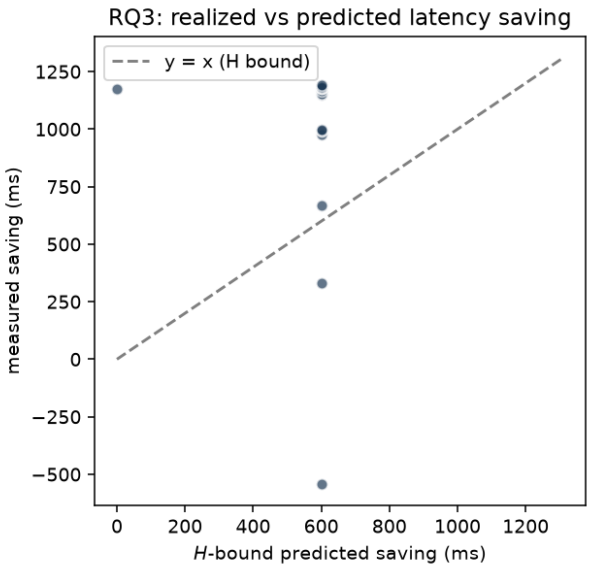


Figure 4: RQ3: per-question measured saving vs. the $H$ bound (60 questions). Most points lie above $y{=}x$ because the pipeline also hides query-generation latency, which $H$ does not model; a few queries where speculation mis-fires show small negative savings.

**Grounding.** $d^\star$ is derived by string matching, so the groundable population excludes non-verbatim answers; reported $\phi_{\text{suf}}$ statistics are conditional on groundability, and we report $t_{\text{sc}}$ as a grounding-free check.

**Retriever and benchmark.** BM25 is lexical and phrasing-sensitive, and results are from a single benchmark; a dense-retriever condition and a second QA set are natural extensions.

# 7 Conclusion

Tool-intent stabilization is a query-intrinsic, model-agnostic quantity that predicts when streaming tool use can help and by how much. On CRAG it is early for most answerable questions and strongly ordered by question type, the $H$ bound is a conservative predictor of realized savings, and the streamable fraction is high at realistic operating points. These results give system builders a principled, training-free basis for trigger design and a build/skip decision for a learned trigger.

# References

[1] Siddhant Arora, Hamza Khan, Kaihang Sun, Xin Luna Dong, Sneha Choudhary, Seungwhan Moon, Xinyao Zhang, Aman Sagar, Sai Teja Appini, Kushal Patnaik, Shubham Sharma, Shinji Watanabe, Anuj Kumar, Ahmed Aly, Yuan Liu, Florian Metze, and Zhaojiang Lin. Stream RAG: Instant and accurate spoken dialogue systems with streaming tool usage. *arXiv preprint arXiv:2510.02044*, 2025.

[2] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, et al. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[3] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.

[4] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[5] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[6] Xiao Yang, Kai Sun, et al. CRAG: Comprehensive RAG benchmark. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.